

Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification - Self-Service Coffee Kiosk System

Architecture Selection: We chose **Layered Architecture** for the **Self-Service Coffee Kiosk** System.

Architectural Choice

The kiosk is split into three layers. The Presentation layer has the User Interface component (touch screen). The Business layer has the Order Manager, Payment Service and Receipt Printer components. The Data layer has the Database component that stores the menu and prices. The components communicate through defined interfaces: Order API, Payment API, Printer Hardware Interface and Database Query Interface.

Two Reasons for the Choice

- 1. The kiosk flow is fixed and simple. The customer picks a coffee (Espresso, Americano or Latte), picks a size (Small or Large), pays by credit card and gets a receipt. This is a straight top-to-bottom flow, which fits layers well: the screen only takes input and shows output, the Order Manager handles the logic and coordinates payment and receipt operations, and the database only stores data.**
- 2. Easy to change one part without touching the rest.** If the printer model is replaced or a new payment method is added later, only that one component changes because the others depend only on its interface. Microservices would be too much for one kiosk (many services, network calls, harder deployment) and Client-Server would need a central server, so the kiosk would stop working if the network or server goes down.

Security Advantage

The UI layer never talks to the database or the payment system directly. Every request goes through the Order Manager, which can validate the input first (only 3 coffees and 2 sizes are allowed). Card details are handled only inside the Payment Service component and are not passed to the UI or stored in the menu database, and the Database Query Interface uses parameterised queries. So even if the touch screen layer is misused, it cannot reach card data or the data layer directly.

Performance Benefit

All layers run on the same kiosk machine, so calls between components are normal in-process method calls with no network delay (unlike microservices). The menu and pricing data is small, so the Order Manager can cache it after startup and the screen responds quickly to touches. The slow tasks (card authorisation and printing) are kept in their own components, so they do not block the UI code.

Trade-off

Layered architecture adds a little overhead because a request passes through several layers, and it is harder to scale one part on its own. This is acceptable here because each kiosk runs its own copy and the load per kiosk is small.